LIDR

Módulo 1 · Los prompts de la sesión

Todo lo que se teclea hoy, en orden, para que puedas seguir la sesión sin copiar del chat y para que puedas repetirla después en tu propio proyecto, que es donde de verdad sirve.

No es el guion del mentor. Aquí está lo que se ejecuta y **por qué cada restricción está donde está**, que es la parte que se pierde cuando alguien copia un prompt suelto.

Antes de empezar

Trabaja sobre **tu propio fork** de:

`https://github.com/LIDR-academy/flowsync-ai4devs`

Fork, no clone directo: la sesión cierra abriendo un PR de verdad, y sobre un clon del repositorio del curso no tienes permiso de escritura. La secuencia es **fork → clone de tu fork →** `git checkout s1/start` **→ instalar dependencias de** `backend/` **y** `frontend/`, y el detalle está en el prework del módulo.

La rama de partida es `s1/start`: el proyecto tal cual, **sin nada configurado**. No tiene `.claude/`, ni `CLAUDE.md`, ni `AGENTS.md`, ni `.mcp.json`. Eso es a propósito: lo que hoy se construye es exactamente eso.

> 🖥️ **Dos proyectos en un repositorio.** FlowSync no tiene `package.json` en la raíz: `npm` se ejecuta dentro de `backend/` (AdonisJS) o de `frontend/` (React + Vite), en dos terminales distintas. Es justo el motivo del último ejercicio del día.

> 🔌 **Sobre el tablero de Jira.** Dos de los prompts de hoy traen un ticket desde Jira por MCP. En el directo ese ticket vive en el tablero del curso, así que **los verás correr en la pantalla del mentor**. Para reproducirlos por tu cuenta necesitas un tablero de Atlassian propio con un ticket equivalente, asignado a ti y en estado **"Por hacer"**: el ticket de hoy es "*Implementar login en el frontend*", consumiendo `POST /api/v1/auth/signup` y `POST /api/v1/auth/login`, con shadcn/ui, redirección a una vista protegida que consuma `GET /api/v1/account/profile`, mensajes de error claros y navegación entre login y registro. Todo lo demás de esta hoja funciona sin Jira.

Darle memoria al proyecto

Lo primero que se configura en un repositorio nuevo. Se arranca `claude` en la raíz del proyecto y:

```
/init
```

El agente explora el repositorio solo y propone un `CLAUDE.md` inicial: stack, estructura, comandos de test y de build. **Es el punto de partida, no el final.** Fíjate en lo que ese archivo generado **no** trae: ni una sola regla de cómo trabaja tu equipo. Esa parte se añade a mano al final de la sesión, y hay un motivo para dejarla para el final.

Conectar una fuente que no está en el repositorio

Hasta aquí todo el contexto salía del propio código. MCP es el estándar con el que el agente se conecta a herramientas de fuera: un ticket, un repositorio remoto, una base de datos. El asistente interactivo:

```
claude mcp add
```

Este comando es el de ejemplo, para ver la forma. El de verdad, con el tablero real detrás, es el siguiente.

El MCP de Jira, con el tablero real

En la carpeta donde estás montando el harness:

```
claude mcp add --transport http --scope project atlassian
https://mcp.atlassian.com/v1/mcp/authv2
```

Las dos banderas importan, y las dos se equivocan solas si no se escriben:

`--transport http`, porque el transporte `sse` que verás en tutoriales viejos está **deprecado**. Si copias uno de esos, conecta hoy y deja de conectar mañana.

`--scope project`, y no el scope local que sale por defecto. Local ata la conexión a tu `~/.claude.json`: funciona en tu máquina y no existe para nadie más. Project la escribe en un `.mcp.json` en la raíz del proyecto, que **sí se versiona con el resto del harness**. Ábrelo y compruébalo: dentro no hay ninguna credencial, solo el tipo de servidor y su URL. Quién se autentica es cosa de cada persona, por OAuth, aparte y en local.

La primera vez que abras `claude` con un `.mcp.json` nuevo te pedirá aprobar el servidor. Es normal.

Las piezas que se invocan: dos skills y un subagente

No son prompts, son archivos del repositorio, y sin ellos los comandos de más abajo no existen. Se crean dentro del proyecto, se versionan, y viajan con el equipo.

`.claude/skills/priority-ticket/SKILL.md`:

```
---
name: priority-ticket
description: Trae el ticket de mayor prioridad asignado al usuario logeado en Jira (vía
MCP) y arranca el trabajo sobre él. Usar al empezar una tarea nueva.
---
# Priority ticket
1. Consulta Jira vía MCP: busca los tickets asignados al usuario actual **en estado "Por
hacer"**, ordénalos por prioridad y toma el de mayor prioridad. No consideres tickets ya
en "En curso" o "En revisión" — evita re-tomar uno que ya está en marcha o cerrado.
2. Resume sus criterios de aceptación.
3. Entra en plan mode y propone cómo implementarlo (sigue las convenciones de
AGENTS.md/CLAUDE.md si existen).
4. En cuanto el usuario apruebe el plan: mueve el ticket a "En curso".
5. Al terminar (con el PR ya creado siguiendo las reglas de CLAUDE.md): mueve el ticket
a "En revisión" y deja un comentario en el ticket con el enlace al PR.
```

Fíjate en lo que esta skill NO dice: nada de crear rama, nada de abrir PR. Eso vive en `CLAUDE.md`, porque aplica siempre, tengas o no un ticket de por medio. La skill solo gestiona lo que es específico de Jira. Es la línea que decide si mañana puedes trabajar sin ella.

`.claude/skills/commit/SKILL.md`:

```
---
name: commit
description: Genera un commit convencional a partir de los cambios staged. Usar al
cerrar una tarea.
---
# Commit convencional
1. Ejecuta `git diff --staged` y resume el cambio.
2. Redacta un mensaje `tipo(scope): descripción` (feat/fix/refactor/docs/test/chore).
3. Commitea directamente con ese mensaje.
```

`.claude/agents/adversarial-reviewer.md`:

```
---
name: adversarial-reviewer
description: Revisa un PR buscando bugs, huecos de seguridad y desviaciones de
AGENTS.md. Su único objetivo es refutar, no aprobar. Read-only.
tools: Read, Grep, Glob, Bash
model: sonnet
---
Eres un revisor escéptico. Intenta ROMPER el cambio, no aprobarlo: busca
edge cases (¿qué pasa con credenciales vacías? ¿con un 401 del backend?),
fugas de seguridad (¿el token queda expuesto en algún log o en la URL?) y
desviaciones de AGENTS.md. Devuelve hallazgos priorizados. No edites archivos.
```

Del subagente, las dos líneas que hacen el trabajo son **"intenta ROMPER el cambio, no aprobarlo"** y **"no edites archivos"**. La primera es lo que evita el revisor complaciente que te felicita por todo. La segunda lo mantiene en su papel: quien revisa no arregla, o nunca sabrás qué estaba mal.

Una pieza que se dispara sola: el hook de formateo

Una skill la invocas tú. Un hook se ejecuta solo, en un punto fijo del ciclo del agente. Esta vez no se escribe el archivo a mano, se le pide:

```
Configura un hook que corra el formateador del frontend cada vez que se edite un
archivo.
```

⚠️ **El agente te va a preguntar, y esa pregunta es parte de la lección.** Descubre que el frontend **no tiene formateador instalado**, solo `oxlint`, que es un linter y no lo mismo. Suele ofrecer instalar Prettier en el frontend, correr `oxlint --fix`, o reutilizar el Prettier que ya existe en el backend. La respuesta de hoy es **instalar Prettier en el frontend**, y si pregunta sobre qué archivos disparar el hook, **`frontend/src/**`**.

Lo que hay debajo: el `CLAUDE.md` que generó `/init` **no dice nada** sobre cómo formatear. Esa decisión no estaba escrita en ningún sitio, y el agente la resuelve leyendo el `package.json` real en vez de inventársela. Un agente bien configurado pregunta cuando debe.

Y una vez puesto, es **determinista**: pase lo que pase, cada edición se formatea. Ahí está la diferencia con pedirlo en el prompt, donde solo sube la probabilidad.

Las reglas de proceso, al final y escritas a mano

Este bloque se pega **al final del `CLAUDE.md`** que generó `/init`, y lo escribes tú, no la IA:

```
## Reglas de proceso
- Antes de tocar código: crear una rama nueva (`git checkout -b feat/<slug>`). Nunca commitear directo en `main`/`s1/start`.
- Al cerrar la tarea: usar la skill `/commit`, luego `gh pr create` con una descripción completa de los cambios en el cuerpo del PR.
- Después de abrir el PR: usar el subagente `adversarial-reviewer` sobre él, antes de darlo por terminado.
- No repitas ese resumen en el chat: la sesión se va a perder, el PR no. Responde solo con la URL del PR.
```

Va al final por una razón, no por orden alfabético: estas reglas nombran piezas (`/commit`, el subagente) que hasta hace cinco minutos no existían. Escribirlas antes habría sido documentar algo inexistente.

Y las escribes tú porque son una **decisión de equipo**, no un hecho del repositorio que un agente pueda deducir explorando. Es la diferencia entre lo que se genera y lo que se decide.

> 💡 **La pregunta honesta que deja la tercera línea.** "Usa el subagente revisor" es una guía, no un guardarraíl: sube la probabilidad de que la revisión ocurra, no la garantiza como haría un hook. ¿Debería ser determinista? Es la pregunta correcta, y hoy se queda abierta.

Después, para que el resto de herramientas del equipo lean lo mismo sin mantener dos archivos:

```
ln -s CLAUDE.md AGENTS.md
```

`CLAUDE.md` es el archivo real. `AGENTS.md` es solo la puerta de entrada estándar que leen Cursor, Copilot o Codex; apunta al mismo contenido, así que no hay nada que sincronizar.

Y el primer commit, el del harness, **a mano**:

```
git add .claude CLAUDE.md AGENTS.md .mcp.json
git commit -m "chore: configura el harness de Claude Code"
```

Usar la skill `/commit` para commitear la creación de la propia skill `/commit` sería un poco circular. Se estrena en el trabajo real, que empieza ahora.

Disparar el trabajo con un solo comando

Con el harness montado y commiteado, así arranca una tarea:

```
/priority-ticket
```

Un comando trae el ticket, resume sus criterios, entra en plan mode y propone cómo implementarlo, **cargando `AGENTS.md` solo, sin que nadie se lo pida**.

🔍 **Mira el plan antes de aprobarlo, porque ahí está lo que hace útil el ejercicio.** El ticket pide un registro de "email + password". El agente va a abrir el validador real del backend y a encontrar que además exige `passwordConfirmation` y `fullName`. **El ticket no lo decía**, porque es un ticket de producto y no una especificación técnica, y ese hueco es real. Suele ofrecer dos caminos: pedir también la confirmación y un nombre opcional, o ceñirse al ticket a rajatabla y rellenar esos campos por detrás sin pedirlos en el formulario. La respuesta de hoy es la primera, **el contrato honesto con el backend**.

Eso es plan mode haciendo su trabajo: explorar el código real antes de asumir, en vez de repetir lo que dice el ticket.

El mismo encargo, sin harness

En una copia del proyecto **sin nada configurado**, la skill no existe, así que hay que escribir la instrucción entera:

```
Busca en Jira el ticket de mayor prioridad asignado a mí, resume sus criterios de aceptación, y entra en plan mode para implementarlo.
```

⚠️ **Este es el punto entero de la sesión, y conviene no leerlo mal.** El agente trae el ticket igual de bien: el MCP es el mismo. Lo que no tiene es **memoria del proyecto**: ninguna guía sobre qué usar para el formulario, ningún criterio sobre qué librerías instalar, ninguna regla que le diga cómo se cierra una tarea aquí. Puede improvisar estilos, instalar lo que le parezca, o no cubrir el caso de error que el ticket pedía.

No es un modelo peor. **Es el mismo modelo sin harness.** Mismo modelo, mismo prompt, mismo ticket, distinto resultado: la única variable que cambió es lo que tú configuraste encima.

Cerrar el trabajo: rama, commit, PR y revisión

Solo en la copia con harness, que es la que tiene reglas que cumplir:

```
git checkout -b feat/login-frontend
```

Se aprueba el plan y se aplica el diff. El hook formatea cada archivo que se toca, sin que nadie lo pida. Después:

```
/commit
```

```
gh pr create
```

Y aquí se cierran tres cosas a la vez **sin teclear ni una más**: el paso 5 de la skill mueve el ticket a "En revisión" y le deja un comentario con el enlace al PR; la regla del `CLAUDE.md` lanza al subagente `adversarial-reviewer` sobre el PR recién abierto; y esa misma regla hace que la respuesta final en el chat sea **solo la URL**, no un párrafo.

> 💡 **Mira el chat cuando termine, y luego mira dónde acabó la información.** No hay resumen en la conversación, porque la conversación se pierde. El resumen está en la descripción del PR, que no se pierde. En el ticket está el enlace. Y el revisor ya está trabajando. Nada de eso se pidió en voz alta: estaba escrito.

El segundo ejercicio: levantar todo con un comando

Levantar FlowSync son cinco comandos en el orden correcto, en dos carpetas. Eso no debería depender de tu memoria.

Antes de pedir nada, **comprueba en qué rama estás**:

```
git branch --show-current
```

Tiene que decir `feat/login-frontend`, no `main`. Si sale `main`, vuelve a la rama del login primero: `main` **no tiene el harness**, y si creas la rama de este ejercicio desde ahí, la pierdes entera para este paso.

```
Antes de nada, confirma en qué rama estás y que tiene .claude/ y AGENTS.md — si no,
avísame antes de seguir. Luego crea una rama nueva desde aquí y, en ella, un Makefile en
la raíz del proyecto con dos targets: setup (instala dependencias de backend y frontend,
prepara el .env, genera la APP_KEY, corre las migraciones) y start (levanta backend y
frontend a la vez). Tiene que funcionar igual en macOS, Linux y Windows vía WSL.
```

Fíjate en **lo que este prompt no dice**: no explica que el backend es AdonisJS, ni que el comando de migraciones es `node ace migration:run`. No hace falta, porque el `AGENTS.md` sigue cargado. Estás aprovechando el harness que montaste hace veinte minutos.

Y fíjate en lo que **sí** dice, que es la restricción que más gente se salta: **"confirma en qué rama estás"**. La regla del `CLAUDE.md` manda crear una rama nueva antes de tocar código, pero **no dice desde dónde**, y por defecto sale de donde esté `HEAD` en ese momento, sea la correcta o no. Una regla puede ser cierta y aun así insuficiente.

Cuando termine:

```
make setup
make start
```

Lo que te llevas

- Un `CLAUDE.md` que empezó generándolo el agente y terminaste tú, con la parte que solo tú podías escribir.
- **Dos skills y un subagente** versionados en el repositorio, que viajan con el equipo y no con tu máquina.
- **Un hook**, que es la diferencia entre pedir algo y garantizarlo.
- **Un MCP** conectado con `--scope project`, versionado y sin credenciales dentro.
- **La misma tarea resuelta dos veces**, con harness y sin él, en tu propia pantalla.
- **El flujo de equipo completo**: rama, commit, PR y una revisión que llega sola.

> **Y la frase para llevarse del día:** el harness explica más varianza que el modelo. Hoy lo has visto con el mismo modelo, el mismo prompt y el mismo ticket, cambiando solo lo que había configurado encima.